Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Ankith U Davey 24011101004

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:
- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:
- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:
- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:
- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:
- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	**Size**	**Purpose**	**Applications**
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose
Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$= 448$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Source Code

The complete source code is available in the accompanying GitHub repository: `https://github.com/Ankith-Davey/Deep-Learning-Lab-Ankith-24011101004/tree/main/Lab%203`

9. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Dataset loaded and verified directly:

- `x_train` shape : $(50000, 32, 32, 3)$
- `y_train` shape : $(50000,)$
- `x_test` shape : $(10000, 32, 32, 3)$
- `y_test` shape : $(10000,)$

Task 2

Implement a convolution layer.
Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Kernel	Padding	Stride	Output Size	Feature Map Shape
3×3	Valid (0)	1	30×30	$(30, 30, 8)$
5×5	Valid (0)	1	28×28	$(28, 28, 8)$
7×7	Valid (0)	1	26×26	$(26, 26, 8)$

Task 3

Study hyperparameters.
Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Stride	**Padding**	**Output Shape ($F = 3$)**
1	Valid	$(30, 30, 8)$
1	Same	$(32, 32, 8)$
2	Valid	$(15, 15, 8)$
2	Same	$(16, 16, 8)$

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Output size: max pooling and average pooling produce identical output shapes for the same kernel and stride, since output size depends only on the pooling window and stride, not on the aggregation function.

Pooling Type	**Final Validation Accuracy (10 epochs)**	**Training Time**
Max Pooling	66.42%	61.2 s
Average Pooling	65.98%	59.4 s

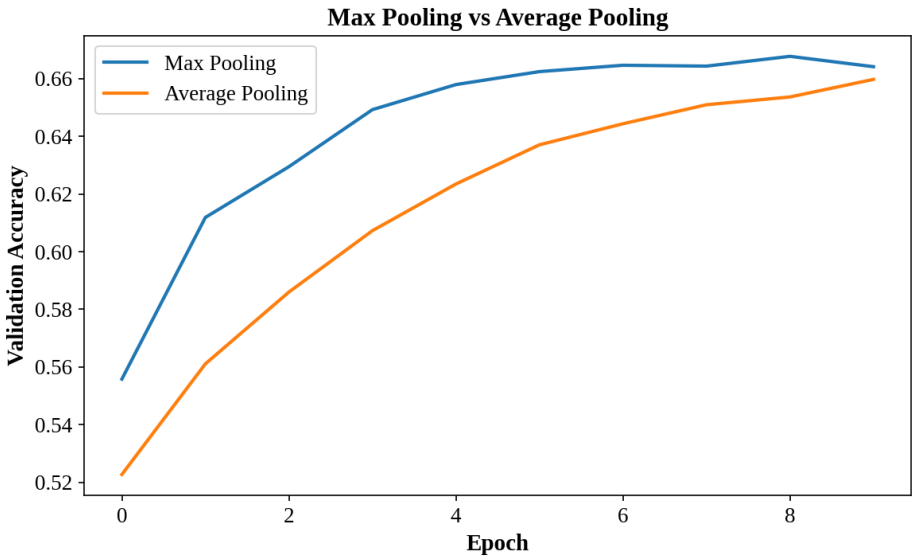

Figure 1: Validation accuracy per epoch: max pooling vs. average pooling.

Task 6
Construct the following CNN.

$$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$$

Train for
- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Total trainable parameters: 25,578.

Task 7
Evaluate
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Metric	**Value**
Test Accuracy	67.45%
Precision (macro)	68.23%
Recall (macro)	67.45%
F1-score (macro)	67.40%

Class	**Precision**	**Recall**	**F1-score**
airplane	0.75	0.64	0.69
automobile	0.80	0.82	0.81
bird	0.52	0.62	0.57
cat	0.55	0.38	0.45
deer	0.54	0.69	0.61
dog	0.58	0.62	0.60
frog	0.72	0.75	0.73
horse	0.69	0.77	0.73
ship	0.83	0.75	0.79
truck	0.84	0.70	0.76

10. Mandatory Plots

Generate
1. Sample Images
2. Class Distribution
3. Training Accuracy
4. Validation Accuracy

5. Training Loss
6. Validation Loss
7. Feature Maps
8. Confusion Matrix

Write a 2–3 line inference for each plot.

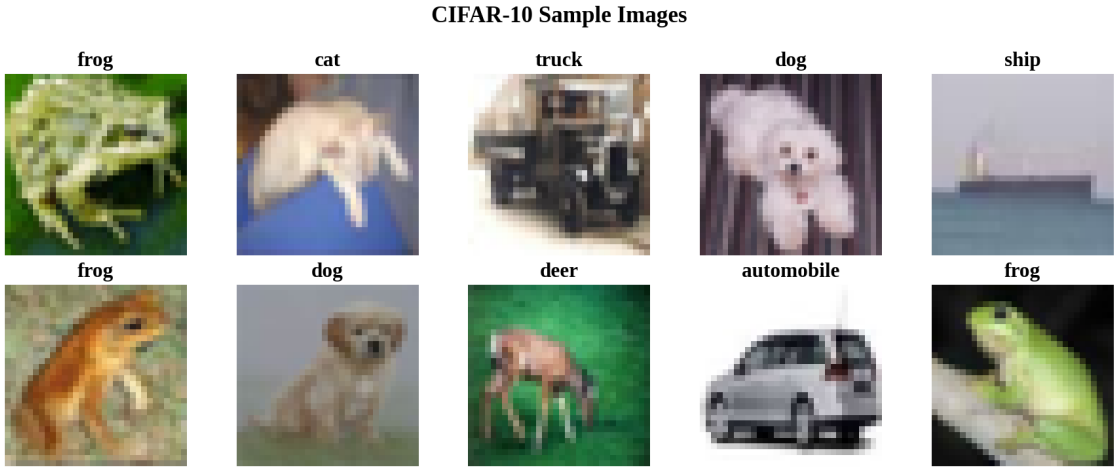

Figure 2: Ten randomly sampled CIFAR-10 training images with their class labels.

The ten sampled images span natural scenes (frog, deer), animals (cat, dog), and vehicles (truck, ship, automobile) at low 32×32 resolution, confirming the dataset loaded correctly with intact labels. Even at this resolution, class-defining shapes remain visually distinguishable to a human eye.

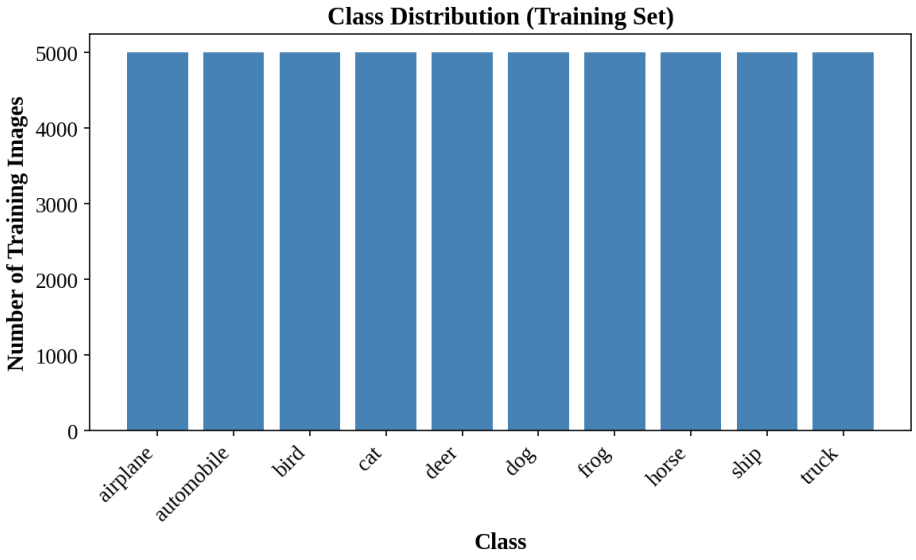

Figure 3: Class distribution across the training set.

All ten classes are represented by exactly 5,000 training images, confirming that the CIFAR-10 dataset is perfectly class-balanced and with no bias towards any class.

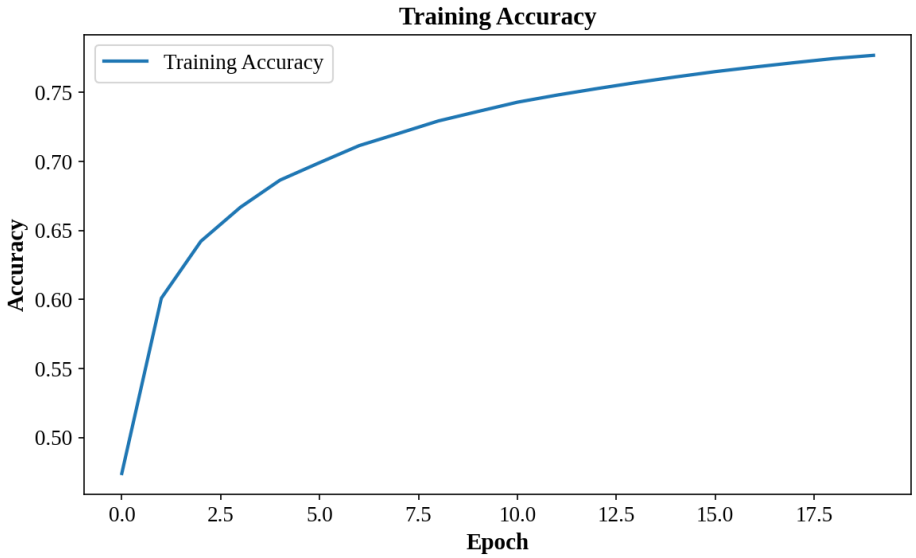

Figure 4: Training accuracy across 20 epochs.

Accuracy rises steadily and near-monotonically across all 20 epochs, reaching 77.69% by the 20th epoch. The absence of any plateau suggests the model was still actively learning the training set when training stopped.

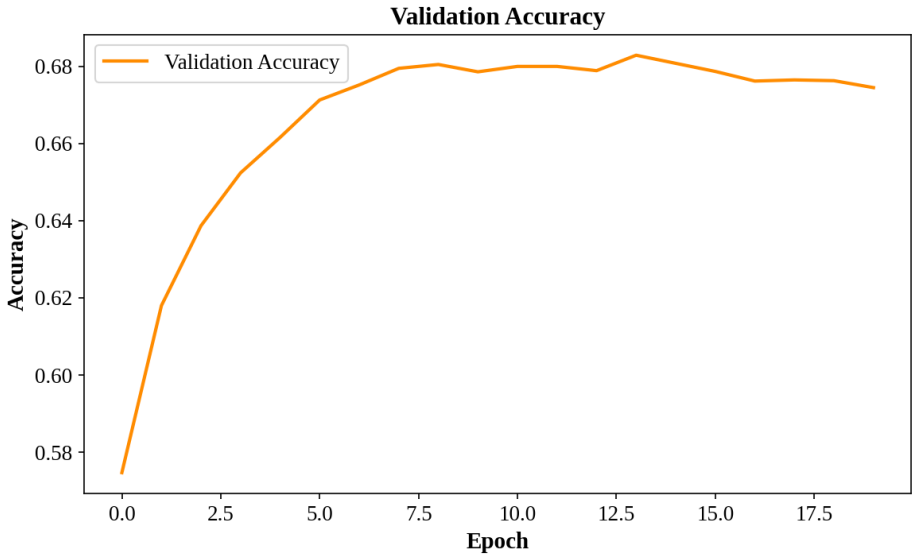

Figure 5: Validation accuracy across 20 epochs.

Accuracy climbs until roughly epoch 12–14 ($\approx 68\%$) and then flattens and drifts slightly downward to 67.45% by epoch 20. This divergence from the still-rising training accuracy curve indicates that the model slightly overfits.

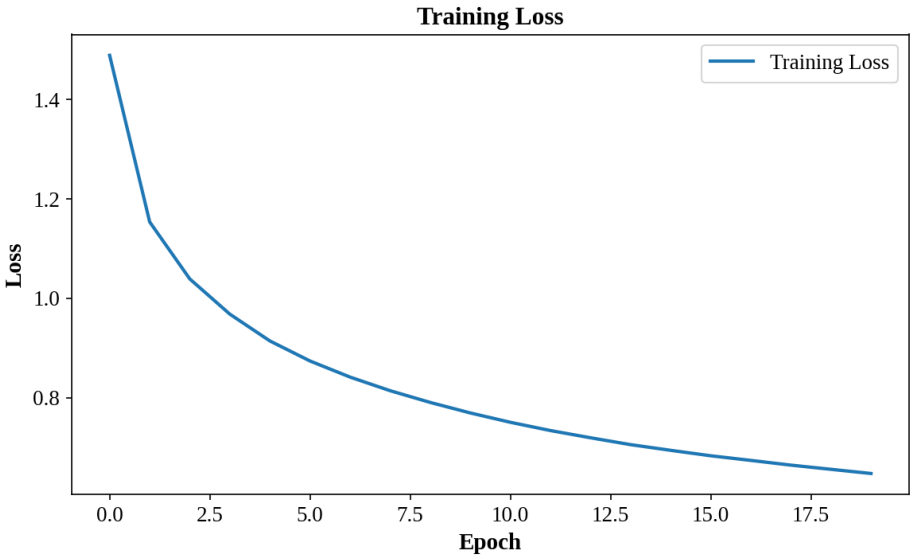

Figure 6: Training loss across 20 epochs.

The training loss decreases smoothly and continuously through all 20 epochs. This confirms the Adam optimizer converged cleanly on the training set at this learning rate and batch size.

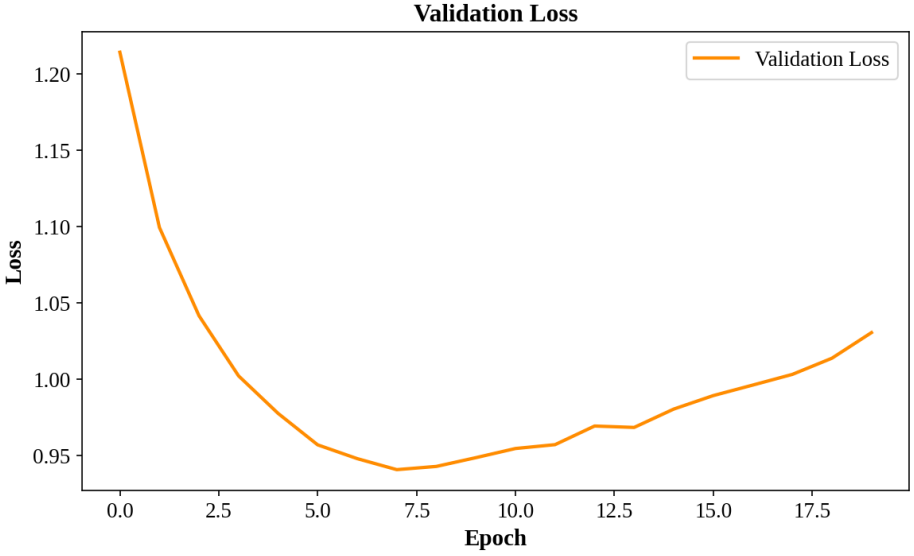

Figure 7: Validation loss across 20 epochs.

The validation loss reaches its minimum around epoch 10–12, but then as the epoch further increases, the validation loss also starts to increase even though the training loss keeps decreasing. This shows a case of overfitting supporting the validation accuracy curve.

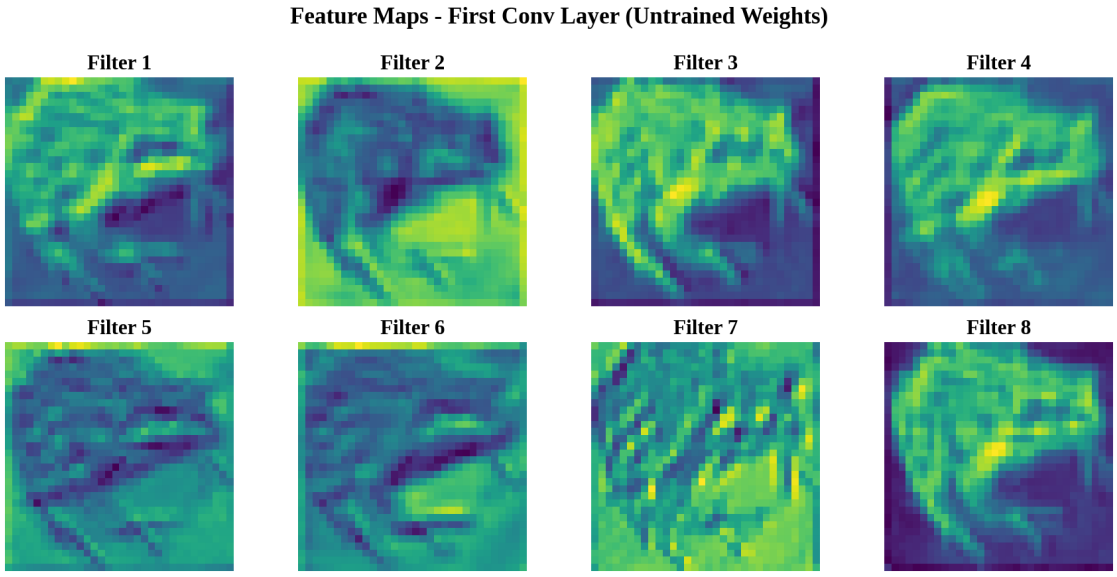

Figure 8: Feature maps from the first convolution layer, untrained (random) weights.

Even with random weights, most of the filters still trace the input image's silhouette and background texture, since a random linear mix of RGB channels partially preserves the edge structure. The maps are not pure noise, but none show the sharp, filter-specific selectivity that the trained weights produce.

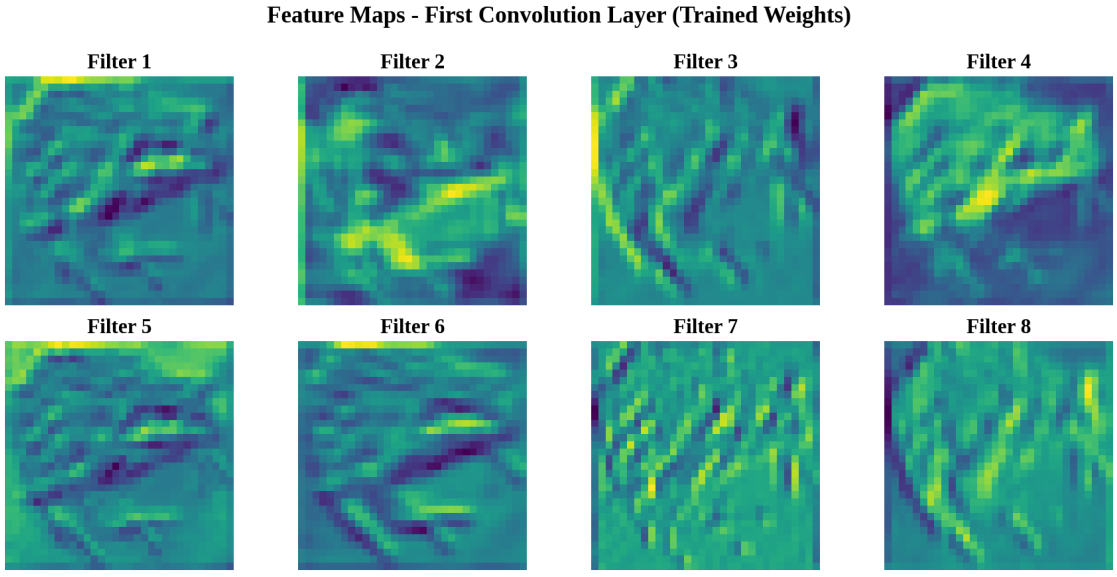

Figure 9: Feature maps from the first convolution layer, after training the CNN classifier.

Several filters now show sharper, thinner responses to specific edges and contours compared compared to the untrained versions (e.g. Filters 3, 6, and 7). This indicates that the first layer learned low-level edge detectors.

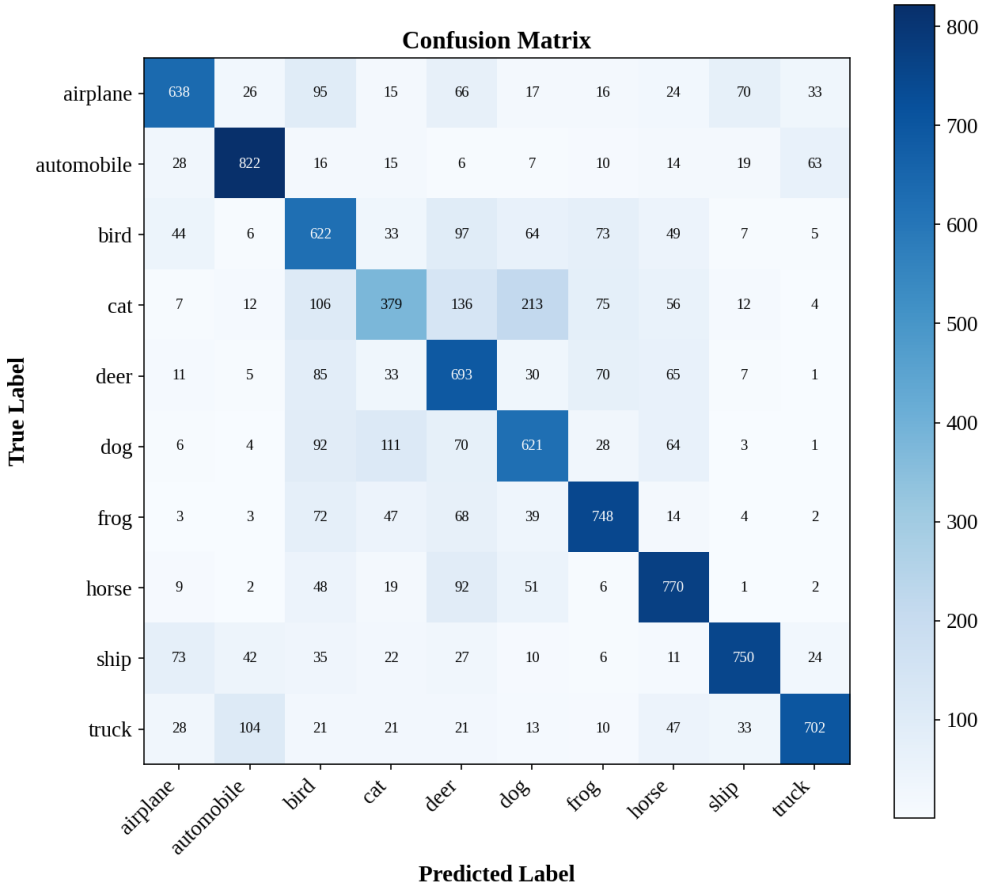

Figure 10: Confusion matrix on the CIFAR-10 test set.

Cat is the weakest class among all the other classes with only 379 correctly classified out of 1,000, and 213 confused as dog and 136 as deer. The vehicle classes show far cleaner diagonals (automobile: 822, ship: 750), confirming errors concentrate among visually similar classes rather than being spread evenly.

11. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

$$\left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = 32$$

   Output size: 32×32.

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

$$(3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1{,}792 \text{ parameters}$$

3. Compare ReLU and Sigmoid activation functions.

Activation	**Final Validation Accuracy (5 epochs)**
ReLU	63.81%
Sigmoid	47.83%

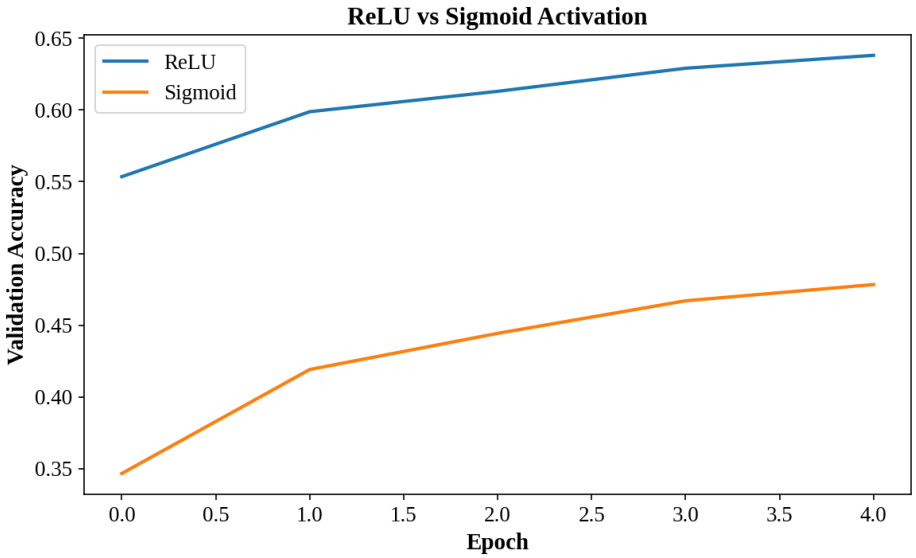

Figure 11: Validation accuracy per epoch: ReLU vs. Sigmoid.

ReLU gives a far higher validation accuracy (63.81%) right from the first epoch as compared to the Sigmoid Activation function which has an accuracy of 47.83%. This is also due to the Sigmoid's vanishing-gradient problem, which slows early-layer learning in a short 5-epoch.

4. Replace Max Pooling with Average Pooling and compare the results.
   Max pooling and average pooling were compared using the same architecture, trained for 10 epochs each. Max pooling reached 66.42% final validation accuracy versus 65.98% for average pooling. Both produced identical output shapes at every pooling layer, since output size under pooling depends only on kernel size and stride, not on the aggregation function used.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

Filter Configuration	**Validation Accuracy (5 epochs)**	**Training Time**
16 / 32 filters	64.59%	39.2 s
64 / 128 filters	68.04%	41.8 s

Increasing filter counts from 16/32 to 64/128, improved validation accuracy slightly by approximately 3.5% within the same 5-epochs, at a modest training-time cost which implies that filters let the network learn a more diverse set of low and mid-level feature detectors in the same number of layers, which also explains the accuracy gain.

12. Results

Record

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score
- Number of Parameters

Metric	Value
Training Accuracy (final epoch)	77.69%
Testing Accuracy	67.45%
Precision (macro)	68.23%
Recall (macro)	67.45%
F1-score (macro)	67.40%
Total Trainable Parameters	25,578

13. Discussion

Answer the following.

1. Why is convolution preferred over fully connected layers for images?
   Convolution layers use a small, spatially-shared kernel that slides across the entire image, so the same set of weights detects a given feature regardless of where it appears in the image. Whereas a fully connected layer assigns an independent weight to every input pixel, which not only increases the number of parameters but also does not capture the spatial information and structure. For a $32 \times 32 \times 3$ image, a single fully connected layer to even a modest hidden size would need significantly more parameters than the 25,578 parameters used by the entire CNN classifier built in this experiment.
2. How does stride affect the feature map size?
   Stride controls how far the kernel moves between applications. In the experiment, a stride of 2 roughly halves each spatial dimension of the output compared to a stride of 1, since the kernel visits half as many positions. Larger strides therefore produce smaller feature maps and reduce the size and the computation but at the cost of skipping some spatial positions the kernel.
3. What is the role of padding?
   Padding adds extra border pixels (generally zeros) around the input before convolution. Its main role, is to control output size. In the experiment, "same" padding is chosen specifically so the output retains the input's spatial dimensions (at stride 1), which also allows deeper stacks of convolution layers without the feature map shrinking away entirely. It also lets the border pixels participate in more convolution windows as compared to "valid" padding where they contribute to comparatively fewer output values.
4. Why is pooling used?
   Pooling is used to reduce the spatial dimensions of feature maps (the max pooling and average pooling both showed that the pooling types roughly halved each spatial dimension), which in turn reduces the number of parameters and computation needed in subsequent layers and provides a degree of translation invariance. It also acts as a mild form of regularization by discarding precise spatial information that is not necessary for classification.
5. How do feature maps represent image characteristics?
   Each filter in a convolution layer learns to respond strongly to a specific visual pattern. An untrained filter's feature map is close to random noise, while a trained filter's feature map highlights the specific regions of the input image where that learned pattern is present. Stacking convolution layers builds increasingly abstract representations. For example the early layers tend to capture low-level features such as edges and colour boundaries, while deeper layers combine these to capture higher-level, more class-specific patterns.
6. Why do CNNs require fewer parameters than MLPs?
   CNNs have less parameters than that of MLPs because unlike MLPs, the CNNs use shared weights. The other major difference betwen CNNs and MLPs is that CNNs make use of the neighbourhood property that is each output depends only on a small neighbourhood of the input and not the

entire image. For example in the experiment, a 3×3 convolution layer with 64 filters over an RGB input needs only 1,792 parameters, regardless of the input's spatial size for a CNN but on the other hand, an MLP layer connecting even a modest number of input pixels to a hidden layer of comparable width would need significantly more parameters, since every input-output pair gets its own independent weight.

14. Conclusion

This experiment implemented and evaluated the core building blocks of a Convolutional Neural Network that is the convolution, pooling, and a full classification pipeline on the CIFAR-10 dataset, using the specified architecture. The resulting model, with 25,578 trainable parameters, reached 77.69% training accuracy and 67.45% test accuracy after 20 epochs. The gap between these two numbers, together with the training/validation loss curves for the main CNN classifier indicate overfitting. So the model's capacity, while small, is spent partly on memorizing training-set specifics rather than generalizable features, particularly after epoch 12–14. The hyperparameter comparisons reinforced the expected theoretical behaviour throughout. The max and average pooling produced identical output shapes while differing slightly in accuracy, ReLU substantially outperformed Sigmoid under a limited 5 epoch budget, and increasing filter count also improved accuracy at a modest computational cost. The per-class evaluation further showed that the visually distinctive classes (automobile, ship, truck) were classified far more reliably than visually similar ones (cat, bird, dog), indicating that the model's errors are concentrated in closely related categories rather than being spread uniformly across all classes. Overall, the experiment met its stated objective of exploring how convolution, activation, and pooling operations combine into a trainable image classifier, while also surfacing a concrete, observable example of overfitting under an unregularized architecture.

15. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.